esercizi 1 aulaweb

1 Esercizio 1

Abbiamo una funzione f per la quale esiste un valore i tale che $f(i) = 0$. Gli algoritmi che seguono cercano un tale valore i, chiamato *punto zero*, dividendo lo spazio di ricerca in due parti: i punti positivi e i punti negativi.

Un algoritmo è corretto se, per **tutte** le esecuzioni, i due processi finiscono dopo che uno dei due ha trovato un punto zero. Per ciascun algoritmo A, B, C e D determinate se è corretto o meno, giustificando la vostra risposta.

Per gli algoritmi C e D usiamo un'istruzione speciale: `await C then R` che viene eseguita in un modo atomico e ha il seguente comportamento: se la condizione `C` è vera, allora il processo esegue `R` e può proseguire la sua esecuzione, se invece la condizione è falsa, il processo viene interrotto e potrà rifare più tardi l'istruzione.

```
bool found=false;
```

```
Process P
int i=0;
while(!found){
  i=i+1;
  found=(f(i)==0);
}
```

```
Process Q
int j=1;
while(!found){
  j=j-1;
  found=(f(j)==0);
}
```

Figura 1: Algoritmo A

```
bool found=false;
```

```
Process P
int i=0;
while(!found){
  i=i+1;
  if(f(i)==0){
    found=true;
  }
}
```

```
Process Q
int j=1;
while(!found){
  j=j-1;
  if(f(j)==0){
    found=true;
  }
}
```

Figura 2: Algoritmo B

```
bool found=false;
int turn=1;
```

```
Process P
int i=0;
while(!found){
  await turn==1 then turn=2;
  i=i+1;
  if(f(i)==0){
    found=true;
  }
}
```

```
Process Q
int j=1;
while(!found){
  await turn==2 then turn=1;
  j=j-1;
  if(f(j)==0){
    found=true;
  }
}
```

Figura 3: Algoritmo C

```
bool found=false;
int turn=1;
```

```
Process P
int i=0;
while(!found){
  await turn==1 then turn=2;
  i=i+1;
  if(f(i)==0){
    found=true;
  }
}
turn=2;
```

```
Process Q
int j=1;
while(!found){
  await turn==2 then turn=1;
  j=j-1;
  if(f(j)==0){
    found=true;
  }
}
turn=1;
```

Figura 4: Algoritmo D

Per tutti gli algoritmi, supponiamo che $f(1)=0$ e $f(j)\neq 0$ per tutti $j \neq 1$

Algoritmo A:

- il processo P si esegue fino a found $= (f(i)==0)$, con $i = i+1 = 1$
- il processo Q si esegue fino a found $= (f(j)==0)$, con $j = j-1 = 0$

P si esegue e mette found a true, Q si esegue e mette found a false perchè $f(j) \neq 0$, quindi nessuno dei due termina.

→ Algoritmo A: non corretto

Algoritmo B:

Se lo scheduler esegue solo Q e il punto zero è positivo, non verrà mai trovato perchè j decrementerà all'infinito.

In generale, partendo con uno dei due processi, andrà in loop e incrementerà / decrementerà all'infinito.

→ Algoritmo B: non corretto

Algoritmo C

- Processo P esegue l'await, quindi turn = 2
- Processo Q esegue il suo ciclo, poi rimette turn = 1 e si blocca sul suo await
- Processo P trova 0, quindi mette found = true ed esce
- Processo Q bloccato nel suo await perchè P non eseguirà più l'await e non metterà più turn = 2

→ Algoritmo C: non corretto

Algoritmo D

Quando un processo esce dal while perchè è stato trovato 0, sblocca l'altro processo per permettere anche a lui di uscire.

→ Algoritmo D: corretto

2 Esercizio 2

Consideriamo la soluzione seguente per il problema della sezione critica proposto dai ricercatori Manna e Pnueli.

```
int wantp=0;
int wantq=0;
```

```
Process P
while(true){
  SNC
  if(wantq==-1){
    wantp=-1;
  }else{
    wantp=1;
  }
  while(wantp==wantq){}
  SC
  wantp=0;
}
```

```
Process Q
while(true){
  SNC
  if(wantp==-1){
    wantq=1;
  }else{
    wantq=-1;
  }
  while(wantq==-wantp){}
  SC
  wantq=0;
}
```

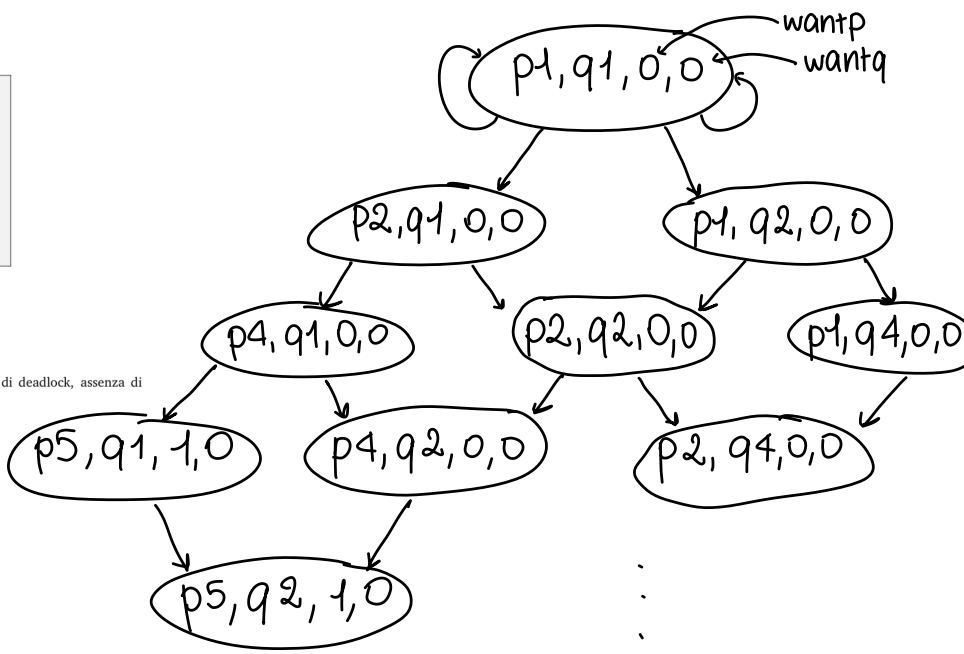
Figura 5: Algoritmo di Manna-Pnueli

1. Costruire il diagramma degli stati per questo algoritmo.
2. Dedurre Quale sono le proprietà verificate da questo algoritmo (mutua esclusione, assenza di deadlock, assenza di starvation con fairness fra i processi)?

```
Process P
while(true){
p1 SNC
p2 if(wantq==-1){
p3   wantp=-1;
   }else{
p4   wantp=1;
   }
p5 while(wantp==wantq){}
p6 SC
p7 wantp=0;
}
```

```
Process Q
while(true){
q1 SNC
q2 if(wantp==-1){
q3   wantq=1;
   }else{
q4   wantq=-1;
   }
q5 while(wantq==-wantp){}
q6 SC
q7 wantq=0;
}
```

1)



2)

- L'algoritmo non verifica la mutua esclusione:

P	Q	wantp	wantq
p1	q1	0	0
p4	q1	0	0
p4	q4	0	0
p4	q5	0	-1
p4	q6	0	-1
p5	q6	1	-1
p6	p6	1	-1

entrambi sono in SC

- L'algoritmo verifica l'assenza di deadlock. Se P e Q sono bloccati, sono bloccati nel while e abbiamo wantp == wantq & wantp == -wantq. Questo è possibile solo se wantp = wantq = 0, ma quando P è in p4 e Q è in q4 abbiamo wantp ∈ {1,-1} e wantq ∈ {-1,1}, quindi wantp ≠ 0 e wantq ≠ 0. Quindi no deadlock.

- L'algoritmo garantisce assenza di starvation

3 Esercizio 3

Supponiamo che abbiamo a disposizione l'istruzione atomica che segue:

```
int flip(){
  atomic{
    x=(x+1)%2; //inversione della variabile globale x
    return x;  //ritorna il nuovo valore
  }
}
```

Consideriamo la seguente soluzione per risolvere il problema della sezione critica.

1. Dimostrate perché questo algoritmo non rispetta la proprietà di mutua esclusione.

```
int x=0;
```

```
Process P
while(true){
  SNC
  while(flip()!=1){
    while(x!=0){}
  }
  SC
  x=0;
}
```

```
Process Q
while(true){
  SNC
  while(flip()!=1){
    while(x!=0){}
  }
  SC
  x=0;
}
```

Figura 6: Algoritmo di mutua esclusione usando `flip`

2. Cosa cambia se nella funzione `flip`, il `%2` è sostituito da `%3`?
3. Per questa ultima soluzione, giustificare se l'algoritmo rispetta le proprietà di assenza di deadlock e di starvation (con ipotesi di fairness fra i processi).

1)

Se P esegue il primo flip(), entra nella sezione critica (SC). Successivamente, Q esegue un flip() e finisce nel ciclo d'attesa while(x!=0). Appena la condizione si sblocca, Q fa un altro flip() ed entra in SC: ora entrambi i processi sono in SC contemporaneamente.

2)

Sostituendo %2 con %3, l'algoritmo rispetta la mutua esclusione. Se P è in SC, significa che x=1. Se Q arriva e chiama flip, il valore diventa x=2 e Q rimane bloccato finchè P non riporta x a zero.

3)

L'assenza di deadlock è dimostrata. Affinchè entrambi i processi siano bloccati, dovrebbero trovarsi simultaneamente nel while(x!=0), ma è impossibile in quanto per arrivare a quel while si deve passare per il while(flip()!=1). Il primo flip() restituisce 1 e manda il processo in SC, mentre il flip() successivo ritorna 2 e blocca quindi solo il secondo processo nel wait(x!=0).

Non rispetta l'assenza di starvation. Ipotizziamo questo scenario:

- P fa flip, x=1 ed entra in SC
- Q fa flip, x=2 quindi entra in attesa
- P esce e mette x=0
- Q non esegue flip e rimane nel while(flip!=1)
- P può rieseguire il flip e rimanere in SC, mentre Q non ci va mai

4 Esercizio 4

Consideriamo la seguente soluzione per risolvere il problema della sezione critica per N processi dove abbiamo un processo S in più che aiuta la gestione.

```
int req=0;
int aut=0;
bool end=true;
```

```
Process P[id] //id in {1,...,N}
while(true){
  SNC
  while(aut!=id){
   req=id;
  }
  SC
  end=true;
  req=0;
}
```

```
Process S
while(true){
  while(req==0){}
  end=false;
  aut=req;
  while(!end){}
  aut=0;
}
```

Figura 7: Algoritmo di mutua esclusione per N processi

1. Dimostrare perché l'algoritmo della figura 7 non rispetta la mutua esclusione.
2. Modifichiamo l'algoritmo per ottenere quello della figura 8. Questo algoritmo verifica la proprietà di mutua esclusione? Qual è il problema di questo ultimo algoritmo?

```
int req=0;
int aut=0;
bool end=true;
```

```
Process P[id] //id in {1,...,N}
while(true){
  SNC
  while(aut!=id){
   req=id;
  }
  SC
  end=true;
  req=0;
  while(aut!=0){}
}
```

```
Process S
while(true){
  while(req==0){}
  end=false;
  aut=req;
  while(!end){}
  aut=0;
}
```

Figura 8: Algoritmo di mutua esclusione per N processi

1)

P1 fa un giro completo aspettando che S metta aut a 1, entra in SC, poi imposta end=true e req=0.
P1 NON ASPETTA CHE S RESETTI LO STATO AUT, quindi rimette req=1 senza che S lo veda e rientra in SC.
Quando S metterà aut a 2, P2 entrerà in SC, ma ci sarà anche P1.

2)

Nella figura 8 si aggiunge while(aut!=0) per obbligare i processi ad aspettare il reset di S. Questo algoritmo introduce però starvation perchè S potrebbe sempre dare l'autorizzazione aut allo stesso processo, ignorando gli altri.